DOCUMENTAÇÃO TÉCNICA · ESTUDO DE CASO

Extrator de Tickets de Táxi

Automação de leitura de recibos com OCR 100% local — arquitetura, bibliotecas, decisões de engenharia e resultados

44 imagens processadas	**~3,5 s** por ticket (serial)
4 binarizações + votação	**0** dados enviados à nuvem

Python 3.13.2 · OpenCV 5.0 · Tesseract 5.5.2 (por.traineddata) · pytesseract 0.3.13 · Tkinter · openpyxl 3.1.5 · PyInstaller

Novembro de 2026

Sumário

01 O problema
O que precisa ser lido, dificuldades reais e por que OCR local

02 Arquitetura da solução
Camadas, módulos, fluxo de dados ponta a ponta

03 O pipeline de OCR em 6 etapas
Da imagem crua ao ticket estruturado

04 Bibliotecas escolhidas e por quê
Comparativo técnico de cada dependência

05 Decisões de engenharia
Votação entre binarizações, IoU, revalidação por recorte

06 Interface e concorrência
Tkinter, threads, multiprocessamento e tempo restante

07 Saída em Excel
openpyxl, fórmula SUM dinâmica e sinalização de erros

08 Distribuição: um único .exe
PyInstaller, Tesseract embutido e modo headless

09 Desempenho e resultados
Medições reais, paralelismo e limites

10 Limitações e próximos passos
Onde o sistema erra e como evoluir com ML

11 Glossário e consulta rápida
Termos, comandos, arquivos e mapa do código

Como ler este documento

Cada seção combina explicação conceitual, trechos reais do código e imagens geradas a partir das próprias fotos processadas pelo sistema. Os quadros coloridos resumem o ponto principal: **azul** = contexto e funcionamento, **verde** = por que a escolha foi boa, **amarelo** = atenção ou limitação, **laranja** = dica prática de uso.

Estrutura de arquivos do projeto

```
projeto_ticket/
├── extrair_tickets.py  — núcleo: OCR, regras, Excel, CLI
├── app_tickets.py      — interface Tkinter
├── TicketsTaxis.spec   — receita do PyInstaller
├── build_exe.ps1       — script de build do .exe
├── img_ticket/         — imagens de entrada
├── README.md           — instruções de uso
└── Documentacao_TicketsTaxis.pdf — este documento
```

01 O PROBLEMA

O objetivo é transformar um lote de fotos de recibos de táxi (tickets térmicos) em uma planilha com duas informações essenciais: o **número do ticket** e o **valor total pago**. Parece simples, mas os recibos reais têm uma combinação de defeitos que atacam justamente os caracteres numéricos — exatamente os que importam.

Desafio	Por que atrapalha
Impressão térmica degradada	Papel térmico perde tinta com o tempo e o calor. Dígitos ficam com traços finos e incompletos (o 8 'abre' e vira 0; o 6 vira 5).
Texto cinza muito claro	Alguns tickets saem com pouquíssima tinta. Uma binarização única (Otsu) apaga a linha inteira e o valor simplesmente desaparece.
Duas vias no mesmo papel	O recibo tem a via do passageiro (VALOR DA CORRIDA + VALOR TOTAL A PAGAR) e a via da empresa, que repete "VALOR R$" mais abaixo. É preciso escolher o valor certo.
Ruído estrutural no lugar de letras	Ícones e figuras à esquerda do texto fazem o OCR trocar as primeiras letras dos rótulos ("PEDÁGIO" → "EDÁGIO"), exigindo tolerância na identificação do rótulo.
Foto com inclinação, sombra e baixa resolução	As imagens vêm de celular, com iluminação irregular e sem garantia de orientação ou distância fixa.
Confusão clássica de dígitos	O → 0, I/l/| → 1, S → 5, B → 8, Z → 2. É a maior fonte de erro em números de ticket.

Por que não usar um serviço de OCR na nuvem?

1. Privacidade. Os tickets contêm dados de corridas; enviá-los a terceiros é um risco desnecessário. **2. Custo.** O volume é pequeno e esporádico — API por chamada não compensa frente a um processo local gratuito. **3. Portabilidade.** O programa precisa rodar em um computador comum do escritório, sem internet, como um único .exe. O OCR local do **Tesseract** resolve os três pontos de uma vez.

Uma janela onde o usuário escolhe a pasta de imagens, acompanha barra de progresso e tempo restante, e ao final recebe uma planilha **.xlsx** pronta, com todos os valores somados por uma fórmula viva. Nada de digitação manual, nada de dependência de internet.

02 ARQUITETURA DA SOLUÇÃO

O projeto foi dividido em **três camadas** com responsabilidades bem separadas. É essa divisão que permite usar o mesmo núcleo de OCR pelo terminal (script) e pela interface gráfica — e empacotar tudo em um único executável.

Camada	Arquivo	Responsabilidade
Núcleo (motor)	`extrair_tickets.py`	Pré-processamento, OCR, interpretação do texto, validação dos valores e geração do Excel. Não conhece interface gráfica e também funciona como CLI.
Interface	`app_tickets.py`	Janela Tkinter: pasta, barra de progresso, tempo decorrido/restante, tabela de resultados e botão para abrir a planilha. Consome o núcleo em threads e processos.
Distribuição	`TicketsTaxis.spec` `build_exe.ps1`	Receita do PyInstaller: embute Python, OpenCV, numpy, Pillow, openpyxl, Tkinter e o próprio Tesseract no .exe, com console=False (sem janela preta).

Fluxo de dados ponta a ponta



▲ Cada etapa é uma função independente do núcleo, o que facilita testar e ajustar uma sem quebrar as demais.

Como as peças se encaixam

A interface nunca faz OCR diretamente: ela chama `nucleo.processar_lote()`, que distribui as imagens entre processos e devolve objetos **Ticket** prontos via callback de progresso. Do outro lado, o modo CLI chama a mesma função com progresso no terminal. Ou seja, existe **uma única implementação** da lógica de leitura.

app_tickets.py — a interface só consome o núcleo

```
import extrair_tickets as nucleo

idioma = nucleo.idioma_disponivel()          # "por" se o tessdata existir
tickets = nucleo.processar_lote(
    arquivos,
    idioma,
    progresso=lambda feitos, total, caminho, ticket: ...
)
caminho_xlsx = nucleo.salvar_excel(tickets)  # planilha pronta
```

Princípio de projeto: o núcleo é puro

Toda a inteligência vive em `extrair_tickets.py`, que não importa Tkinter. Isso significa que a lógica pode ser testada em scripts, reutilizada em outra interface ou rodada em servidor sem mudar uma linha. A interface é apenas uma 'casca' fina.

03 O PIPELINE DE OCR EM 6 ETAPAS

Esta é a parte central da automação. Cada imagem percorre seis etapas; qualquer erro cometido em uma delas se propaga. Por isso o desenho prioriza **redundância** (várias tentativas de leitura) seguida de **consenso**.

Etapa 1 — Leitura robusta do arquivo

A imagem é carregada em tons de cinza. Detalhe importante: usa-se `np.fromfile` + `cv2.imdecode` em vez de `cv2.imread`, porque o imread do OpenCV falha silenciosamente com caminhos que contêm acentos (comum no Windows: "Área de Trabalho", "Faturamento").

Etapa 2 — Normalização de escala e 4 binarizações

Antes de qualquer OCR, o sistema estima o tamanho do texto usando componentes conexos e amplia a imagem para que o texto fique com cerca de 60 px de altura — o Tesseract é muito sensível a texto pequeno. A ampliação é limitada a 1700 px de lado, pois acima disso o OCR só fica mais lento, sem ganho de acurácia.

Em seguida, a imagem é binarizada de **quatro** maneiras diferentes. Cada uma favorece um tipo de defeito; depois o sistema vota entre elas.

Variante	Como é gerada	Para que serve
otsu	Limiar automático global (Otsu)	Tickets nítidos, texto escuro normal. É a base de todas as outras.
open	Otsu + abertura morfológica (kernel 2×2)	Remove grão e 'cabelinhos' do papel térmico, regularizando dígitos degradados.
soft	Blur leve + limiar fixo suave (205)	Preserva texto cinza muito claro que o Otsu apagaria por completo.
clahe_open	CLAHE (contraste local) + threshold adaptativo + abertura	Resgata texto apagado em regiões com iluminação desigual.

Veja o efeito nas imagens reais (ticket com tinta degradada):



▲ As quatro binarizações aplicadas ao mesmo ticket. Repare como a variante 'soft' mantém traços que o 'otsu' elimina — é isso que permite ler recibos apagados.

Etapa 3 — OCR com caixas por palavra

Cada binarização é enviada ao Tesseract com `--psm 6` (assume um bloco uniforme de texto) e `preserve_interword_spaces=1` (mantém os espaços entre colunas, essencial para separar rótulo de valor). O idioma é **por** (português). Usa-se `image_to_data` para obter, além do texto, a **caixa** (x, y, w, h) e a **confiança** de cada palavra.

extrair_tickets.py — OCR configurado para recibos

```
dados = pytesseract.image_to_data(
    img, lang=idioma,
    config="--psm 6 -c preserve_interword_spaces=1",
    output_type=pytesseract.Output.DICT,
)
for i, bruto in enumerate(dados["text"]):
    texto = (bruto or "").strip()
    conf  = float(dados["conf"][i])
    if not texto or conf < 20:      # descarta ruído de baixa confiança
        continue
    candidatos.append({... "conf": conf, "fonte": nome_da_binarizacao})
```

Etapa 4 — Consenso entre as leituras

As quatro binarizações podem ler o mesmo número de quatro formas diferentes. Em vez de escolher uma, o sistema reúne todas as leituras e resolve em três passos:

Passo	Regra	Efeito prático
1. Ordenar	Todas as palavras por confiança decrescente	A leitura mais confiável é considerada primeiro.
2. Deduplicar	Descarta leituras que representam o mesmo símbolo físico, comparando as caixas por **IoU** (interseção sobre união) ≥ 0,35 ou dígitos iguais em posição muito próxima	Uma única palavra no papel vira uma única entrada, mesmo que as 4 binarizações a tenham lido 4 vezes.
3. Agrupar em linhas	Agrupa por centro vertical, com tolerância baseada na **altura mediana** do documento	Não funde "VALOR DA CORRIDA" com "VALOR TOTAL" quando um token sai com caixa exagerada.

extrair_tickets.py — deduplicação por IoU

```
candidatos.sort(key=lambda p: -p["conf"])   # melhores primeiro
aceitas = []
for palavra in candidatos:
    if any(_mesmo_simbolo(palavra, outra) for outra in aceitas):
        continue                            # já temos essa palavra
    aceitas.append(palavra)                 # vence a de maior confiança
```

Etapa 5 — Regras de negócio: dar sentido ao texto

Com as linhas reconstruídas, entra a parte que não é visão computacional, e sim conhecimento do domínio do recibo. O sistema:

- **Identifica o rótulo** de cada linha com expressões regulares tolerantes a erros de OCR: `v[a4][l1i][o0]r` encontra VALOR mesmo lendo "VA4LOR" ou "VAL0R"; `t[o0]t[ai1l]` encontra TOTAL; `c[o0]rr[i1l]da` encontra CORRIDA.
- **Separa as seções** do recibo usando a linha que contém PAGAMENTO como divisor: o que está acima é a corrida; abaixo é o pagamento.
- **Escolhe o VALOR certo**: entre vários "VALOR", pega o que aparece imediatamente antes do primeiro "VALOR TOTAL A PAGAR" (a via do passageiro).
- **Lê valores da coluna da direita** (x ≥ 55% da largura), dando prioridade a números com vírgula decimal explícita — sem isso, "124,00" + "144" poderia virar 112500.00.
- **Descarta datas e horas** ao procurar valores, para não confundir "12/03/2026" com dinheiro.

- **Corrige dígitos** do número do ticket: O→0, I/l/|→1, S→5, B→8, Z→2, G→6.
- **Rejeita falsos positivos**: números parecidos com ano (2020–2030) não são aceitos como número de ticket.
- **Descarta leituras de baixa confiança**: abaixo de 55 (número) ou 45 (valor), em vez de arriscar, o campo vira "N/ IDENTIFICADO".

extrair_tickets.py — validação aritmética do ticket

```
esperado = round(valor_corrida + bagagem + pedagio, 2)
if abs(esperado - total) > 0.01:
    # o total não fecha: relê o recorte da linha com vários limiares
    releitura = reler_valor(totais[0].recorte)
    if releitura is not None and abs(releitura - esperado) < 0.01:
        ticket.total = releitura
        ticket.observacoes.append("Total relido por recorte")
```

A regra mais poderosa: aritmética como validador

Um recibo é internamente consistente: **VALOR DA CORRIDA + PEDÁGIO + BAGAGEM = TOTAL**, e o TOTAL deveria bater com o valor pago. Quando a leitura do OCR fura essa igualdade, há erro — o sistema detecta, recorta só aquela região, relê com cinco limiares diferentes em duas escalas e aceita a leitura que fecha a conta. É checagem de integridade aplicada a OCR.

Etapa 6 — Objeto Ticket e saída

O resultado de tudo isso é um objeto **Ticket** (dataclass) com campos tipados: número, valor da corrida, pedágio, bagagem, total, valor pago, data/hora, forma de pagamento, origem, confiança e uma lista de observações explicando cada correção feita. Esse objeto alimenta o terminal, o JSON opcional e a planilha.

extrair_tickets.py — o contrato de saída

```
@dataclass
class Ticket:
    arquivo: str
    numero: str | None = None
    valor: float | None = None
    valor_corrida: float | None = None
    total: float | None = None
    data_hora: str | None = None
    forma_pagamento: str | None = None
    origem: str = ""
    confianca: str = "baixa"
    observacoes: list[str] = field(default_factory=list)
```

04 BIBLIOTECAS ESCOLHIDAS E POR QUÊ

Toda a pilha foi escolhida com um critério em mente: **rodar 100% offline, em Windows, dentro de um único .exe**. Isso elimina dependências pesadas de nuvem e favorece bibliotecas maduras, com pacotes binários prontos para Windows.

Biblioteca	Versão	Papel no projeto
Python	3.13.2	Linguagem-base. Ecossistema completo de visão computacional e excelente suporte do PyInstaller.
OpenCV (opencv-python)	5.0.0	Pré-processamento: tons de cinza, threshold, morfologia, CLAHE, componentes conexos, resize. Também decodifica imagens de forma segura com caminhos acentuados.
numpy	2.1.3	Base numérica do OpenCV. Usado para medir a mediana de alturas de texto e lidar com os pixels.
Tesseract (OCR)	5.5.2	Motor de OCR propriamente dito. Roda em processo separado, o que permite paralelismo real.
pytesseract	0.3.13	Ponte Python → tesseract.exe. Dá acesso ao modo TSV/DICT com caixas e confianças por palavra.
Tkinter	nativo	Interface gráfica. Já vem no Python, zero dependências extras — decisivo para o .exe ficar simples.
openpyxl	3.1.5	Gera o .xlsx com fórmulas, cores, bordas e mesclagem de células.
PyInstaller	6.x	Empacota tudo em um .exe de arquivo único, sem console.

Por que OpenCV e não Pillow ou scikit-image?

Critério	OpenCV	Pillow	scikit-image
Velocidade em operações por pixel	Altíssima (código C++ otimizado, uso de SIMD)	Boa, porém limitada a operações básicas	Boa, mas mais lenta em loops grandes
Binarização adaptativa, CLAHE, morfologia	Nativo e rápido	Não tem nativo	Tem, mais lento
Componentes conexos com estatísticas	Uma chamada (connectedComponentsWithStats)	—	Sim
Leitura de imagem com caminho acentuado	Resolvida com imdecode	Sim	Via Pillow
Tamanho no pacote final	Médio (mas necessário)	Pequeno	Grande (depende de numpy/scipy)

Pillow ainda é usado — mas apenas para a interface (exibir ícones no Tkinter) e não no processamento pesado.

Por que Tesseract e não EasyOCR / PaddleOCR / OCR de nuvem?

Opção	Prós	Contras para este projeto
Tesseract 5 (escolhido)	Leve, offline, rápido, gratuito, ótimo em texto impresso nítido, modelo português pronto, fácil de embutir no .exe	Sofre com texto muito degradado — mitigado pelas 4 binarizações e pela validação aritmética

Opção	Prós	Contras para este projeto
EasyOCR	Bom em cenas complexas e inclinadas	Depende de PyTorch (centenas de MB), lento em CPU, .exe gigante
PaddleOCR	Excelente acurácia	Dependências pesadas e complicadas de empacotar no Windows
OCR de nuvem	Reconhecimento de ponta	Requer internet, custo por chamada e envio de dados sensíveis a terceiros

Uma decisão de arquitetura que muda tudo: Tesseract como processo externo

O pytesseract não 'importa' o motor de OCR — ele chama **tesseract.exe** como subprocesso. Isso tem duas consequências valiosas: (1) o Python não fica preso ao GIL durante o OCR, então vários processos podem rodar OCR em paralelo de verdade; (2) é trivial embutir o tesseract.exe e sua pasta tessdata dentro do .exe e apontar o caminho na inicialização.

Em vez de exigir instalação no computador do usuário, o programa procura o tesseract.exe em uma lista de locais — inclusive dentro do pacote PyInstaller — e define as variáveis de ambiente corretas antes de qualquer OCR.

extrair_tickets.py — localização automática do Tesseract

```
candidatos_exe = [
    base / "Tesseract-OCR" / "tesseract.exe",     # ao lado do .exe (portátil)
    recursos / "Tesseract-OCR" / "tesseract.exe", # dentro do .exe (_MEIPASS)
    Path(r"C:\Program Files\Tesseract-OCR\tesseract.exe"),
]
exe = next((str(c) for c in candidatos_exe if c.is_file()),
           shutil.which("tesseract"))
pytesseract.pytesseract.tesseract_cmd = exe
os.environ["TESSDATA_PREFIX"] = str(tessdata_dir)
```

05 DECISÕES DE ENGENHARIA

Esta seção reúne as escolhas que mais influenciaram a acurácia e o desempenho — e o raciocínio por trás de cada uma. São bons pontos de partida para estudos futuros.

1 Votar entre 4 binarizações em vez de escolher uma

Motivo. Cada binarização 'vê' o papel de um jeito. O consenso por confiança + IoU combina o melhor de todas: onde o Otsu falha, o soft pode acertar, e a deduplicação evita contar a mesma palavra várias vezes. **Efeito.** Acurácia bem maior em tickets apagados; custo de ~4× o tempo de OCR por imagem — aceitável para lotes pequenos.

2 IoU como critério de 'mesma palavra'

Motivo. Caixas do mesmo símbolo (fundamentalmente) se sobrepõem muito; caixas de símbolos distintos não. O limiar de 0,35 é tolerante o bastante para pequenas variações de binarização. **Efeito.** Elimina duplicatas sem precisar de OCR pós-processado manual.

3 Confiança mínima e 'Ñ IDENTIFICADO'

Motivo. É melhor admitir que não leu do que registrar um número errado. Erros em planilha de faturamento são caros. **Efeito.** Nenhum valor é inventado; a planilha destaca em cor os casos não lidos.

4 Aritmética do ticket como validador

Motivo. A relação corrida + extras = total é uma verificação de integridade gratuita. Quando falha, há suspeita concreta de erro de leitura. **Efeito.** Corrige trocas como '2' → '7' que nenhuma regra sintática pegaria.

5 Multiprocessamento, não multithreading, para o OCR

Motivo. O gargalo é o tesseract.exe. Vários processos independentes escalam quase linearmente com os núcleos; threads compartilhariam apenas o GIL. **Efeito.** Corte de ~3,5 s para ~1 s por ticket em lotes grandes.

6 Limitar 0,55 × largura para a coluna de valores

Motivo. Nos recibos, os valores estão sempre alinhados à direita. Restringir a busca evita ler números do miolo do texto (placa, telefone, código de autorização). **Efeito.** Menos falsos positivos sem perder nenhum valor real.

7 Ampliar o texto até 60 px, com teto de 1700 px

Motivo. O Tesseract precisa de altura de texto suficiente; acima de certo ponto, ganho zero e custo alto. **Efeito.** Equilíbrio entre acurácia e velocidade.

8 console=False no .exe

Motivo. O tesseract.exe é chamado de dentro do programa; sem cuidado, cada chamada abriria uma janela de console piscando. **Efeito.** Execução silenciosa via STARTUPINFO + CREATE_NO_WINDOW.

Como o erro é tratado: exemplo real

No ticket com tinta degradada, a linha VALOR DA CORRIDA saiu como "4U,00" — leitura que daria 4,00. Mas o TOTAL (76,00) batia com o valor pago. A conta 76,00 − extras = 76,00 apontava para valor de corrida 76,00. O sistema então recortou apenas aquela região, releu em 2 escalas × 5 limiares e obteve 76,00, fechando a conta. O ticket foi salvo com observação: 'VALOR relido por recorte'.

06 INTERFACE E CONCORRÊNCIA

A janela é construída com **Tkinter/ttk**. A regra de ouro de qualquer interface com processamento pesado: a **thread da interface nunca faz o trabalho**. Se fizesse, a janela congelaria e o Windows exibiria "Não responde".

Arquitetura da janela

Camada	Papel
Thread principal (Tk)	Desenha a janela, responde a cliques e lê uma **fila** a cada 120 ms para atualizar progresso.
Thread de trabalho	Chama o núcleo e envia mensagens para a fila ('total', 'progresso', 'concluido', 'erro') — nunca toca em widgets.
Processos	O núcleo distribui as imagens entre vários processos (um por núcleo, reservando um para a interface), cada um rodando seu tesseract.exe.

Comunicação interface ↔ trabalho

A comunicação é unidirecional e não bloqueante: a thread de trabalho coloca eventos em uma `queue.Queue`; a thread principal os consome com `after(120, self._drenar_fila)`. Assim, nenhuma atualização de tela acontece fora da thread do Tk — evitando travamentos e erros clássicos de Tkinter.

app_tickets.py — fila + after = interface fluida

```
def _trabalho(self):
    def enviar(tipo, **dados):
        self._fila.put((tipo, dados))       # worker só enfileira

    enviar("total", total=total)
    nucleo.processar_lote(
        arquivos, idioma, processos=...,
        progresso=lambda f, t, arq, tk: enviar("progresso", feitos=f, total=t, ticket=tk),
    )
    enviar("concluido")

# thread principal:
self.after(120, self._drenar_fila)          # consome e atualiza widgets
```

Estimativa de tempo restante

Mostrar apenas "x de y" é pouco útil. O sistema mantém uma lista de marcos (quantos concluídos e quando) e calcula a **média móvel** de segundos por ticket, estimando o restante como "3m 12s". Como o tempo varia conforme a qualidade da imagem, a média é recalculada a cada ticket — a previsão melhora com o andamento.

app_tickets.py — duração formatada e média móvel

```
def formatar_duracao(segundos: float) -> str:
    segundos = max(0, int(round(segundos)))
    horas, resto = divmod(segundos, 3600)
    minutos, segs = divmod(resto, 60)
    if horas:   return f"{horas}h {minutos:02d}m {segs:02d}s"
    if minutos: return f"{minutos:02d}m {segs:02d}s"
    return f"{segs}s"
```

Por que reservar um núcleo

Usar 12 processos de OCR em 12 núcleos deixa a interface com fome de CPU e ela parece travada. Reservar um núcleo (`processos = os.cpu_count() - 1`) mantém a janela responsiva sem afetar o tempo total.

07 SAÍDA EM EXCEL COM OPENPYXL

A planilha não é só um despejo de linhas: ela foi desenhada para ser usada no dia a dia. Tem título, total fixo no topo, pares (Nº | VALOR) lado a lado, linhas zebradas, valores em negrito e destaque de cores para problemas.

Tkinter: por que não Qt ou uma interface web?

Tkinter vem **dentro** do Python: nenhum pacote extra, nenhuma DLL externa e um .exe final bem menor do que com PyQt/PySide. Para uma barra de progresso, uma tabela e três botões, o ttk entrega visual nativo do Windows sem complicar a distribuição.

Layout da planilha

Elemento	Posição	Detalhe
Título "RELATÓRIO DE TICKETS"	linha 1 (mesclada)	fundo azul-escuro, texto branco em negrito
VALOR TOTAL	linha 3 (congelada)	gravado como **fórmula SUM**, não como número — soma qualquer valor digitado depois
Cabeçalho Nº DO TICKET | VALOR	linha 5	5 pares por linha, azul-claro
Dados	linhas 6+	linhas alternadas em cinza-claro (zebra)
Linhas extras em branco	3 no fim	espaço para lançamentos manuais

Um detalhe importante: fórmula, não valor

Se o total fosse gravado como número fixo, qualquer lançamento manual ficaria de fora. O openpyxl permite gravar a fórmula, e o Excel a recalcula ao abrir. Assim, o total permanece sempre correto mesmo que o usuário acrescente valores à mão.

extrair_tickets.py — fórmula SUM dinâmica

```
parcelas = [f"{get_column_letter(col)}{LINHA_PRIMEIRA_DADO}:"
                       f"{get_column_letter(col)}{LINHA_ULTIMO_DADO}"
                       for col in colunas_valor]
cel_total = ws.cell(row=3, column=4, value="=SUM(" + ",".join(parcelas) + ")")
cel_total.number_format = 'R$ #,##0.00'
cel_total.font = Font(bold=True, size=12, color="FFFFFF")
```

Cores com significado

Amostra	Significado	Ação esperada
	Laranja-suave #F8CBAD "N IDENTIFICADO" — OCR não conseguiu ler	conferir à mão
	Amarelo-suave #FFE699 Confiança média — leitura duvidosa	revisar se possível
	Sem cor #FFFFFF Leitura confiável	nenhuma

Por que openpyxl

É a biblioteca pura Python mais completa para **escrever** .xlsx: suporta fontes, cores, bordas, mesclagem, largura de coluna, congelamento de painéis, formato de moeda e, principalmente, **fórmulas**. Alternativas como pandas exigiriam trazer a base toda e não dariam o mesmo controle visual; o xlsxwriter é ótimo, mas só escreve (não lê), o que limita extensões futuras.

08 DISTRIBUIÇÃO: UM ÚNICO .EXE

O destinatário final não tem Python instalado e não deveria precisar instalar nada. A entrega é um único arquivo **TicketsTaxis.exe** que contém o interpretador, as bibliotecas, as fontes e o próprio Tesseract-OCR.



O que a receita do PyInstaller faz

- **Embute o Tesseract inteiro**: copia tesseract.exe, todas as DLLs e apenas os idiomas usados (por, eng, osd) para dentro do pacote — mantendo o .exe enxuto.
- **Declara imports ocultos** (cv2, numpy, openpyxl, PIL.Image, pytesseract), que o PyInstaller não detecta sozinho.
- **Exclui o que não é usado** (matplotlib, pandas, scipy, PyQt, IPython), reduzindo o tamanho final.
- **console=False**: o .exe abre só a janela, sem console preto atrás.
- **Modo onefile**: tudo dentro de um arquivo; na execução o PyInstaller extrai em uma pasta temporária (_MEIPASS).

O detalhe que evita 'janelinhas piscando'

No modo console=False, cada chamada ao tesseract.exe abriria uma janela de console por um instante. A solução é informar ao sistema operacional que o subprocesso não deve criar janela:

extrair_tickets.py — executar o Tesseract sem console

```
_CRIAR_SEM_JANELA = 0x08000000  # CREATE_NO_WINDOW

def _tesseract_sem_janela(*args, **kwargs):
    info = subprocess.STARTUPINFO()
    info.dwFlags |= subprocess.STARTF_USESHOWWINDOW
    kwargs["startupinfo"] = info
    kwargs["creationflags"] = _CRIAR_SEM_JANELA
    return subprocess.Popen(*args, **kwargs)

pytesseract.pytesseract.subprocess = ...  # patch aplicado na inicialização
```

Build

O build é disparado por `build_exe.ps1` ou diretamente por `pyinstaller --noconfirm TicketsTaxis.spec`. O resultado fica em **dist\TicketsTaxis.exe** e pode ser copiado para qualquer computador Windows 10/11 sem instalação.

Portabilidade do Tesseract

O .exe procura o tesseract.exe em quatro lugares, em ordem: ao lado do próprio .exe (pasta Tesseract-OCR), dentro do pacote (_MEIPASS), e nas pastas padrão do sistema. Assim, se você quiser trocar a versão do motor ou adicionar um idioma, basta colocar uma pasta Tesseract-OCR ao lado do executável — sem reconstruí-lo.

Em máquinas Anaconda, o tessdata costuma ficar em `anaconda3\share\tessdata` — por isso o programa também testa essa localização (pasta pai / share / tessdata).

09 DESEMPENHO E RESULTADOS

As medições abaixo foram feitas neste computador, com o lote real de 44 imagens do projeto. Elas servem de referência para dimensionar expectativas.

Ambiente de medição

Item	Valor
Processador	Intel Core i5-11400 @ 2,60 GHz (6 núcleos / 12 threads, até 4,4 GHz)
Memória	16 GB DDR4
Sistema	Windows 11 · Python 3.13.2 (conda-forge) · Tesseract 5.5.2 (Leptonica 1.87)
Lote de teste	44 imagens de tickets (jpg), densidade de texto variável

Tempo medido (modo serial, 1 processo)

Amostra	Tamanho da imagem ampliada	Tempo	Resultado
N_apagado.jpg	1700 × 914 px	3,51 s	Valor 76,00 (número não lido)
V_apagado.jpg	1700 × 914 px	4,21 s	Valor 101,00
tickets001.jpg	1700 × 914 px	3,39 s	Ticket 19725 · R$ 95,00

Média: ~3,5 s por ticket em modo serial. Em paralelo (`os.cpu_count() - 1` = 11 processos), o lote de 44 imagens leva cerca de 44 × 3,5 ÷ 11 ≈ **~14 segundos**.

Por que o paralelismo escala bem aqui

O custo dominante é o tesseract.exe — um processo separado. Ao rodar vários ao mesmo tempo, cada núcleo da CPU fica ocupado com seu próprio motor de OCR, sem disputa pelo GIL do Python. Diferente de uma tarefa puramente Python em threads, aqui o ganho é praticamente linear até o número de núcleos físicos.

Processos	Tempo estimado (lote de 44)	Ganho
1 (serial)	~2 min 34 s	1×
4	~39 s	4×
11 (padrão)	~14 s	~11×

Onde está o tempo: OCR, não Python

Das quatro binarizações, a chamada ao Tesseract consome a maior parte do tempo. Isso explica duas escolhas: (1) ampliar a imagem até o máximo útil (1700 px) em vez de gigantes; (2) paralelizar por processos. Otimizações no código Python isoladamente teriam impacto pequeno.

10 LIMITAÇÕES E PRÓXIMOS PASSOS

Ser honesto sobre os limites é parte da engenharia. Abaixo, o que o sistema ainda não faz bem e caminhos concretos de melhoria.

Limitação atual	Impacto	Caminho de melhoria
Ticket muito borrado / fora de foco	Notifica "N IDENTIFICADO"	Desfoque de movimento é difícil de reverter; exigir nova foto com app de orientação
Inclinação (skew) forte	Linhas agrupadas de forma errada	Detecção de ângulo (Hough ou minAreaRect) + deskew antes do OCR
Outros layouts de recibo	Rótulos/variantes fora das regex atuais	Ajustar ou externalizar as regex em um arquivo de configuração
Números muito degradados	Troca de dígitos (2↔7, 6↔5)	Pós-processamento de recorte por caractere com validação
Só Windows testado	Outros SOs não validados	Empacotar com PyInstaller para Linux/macOS (o núcleo é multiplataforma)

Ideias de evolução, em ordem de custo-benefício

1. **Deskew automático.** Detecta a inclinação do papel e corrige antes de binarizar. Baixo esforço (poucas linhas de OpenCV) e grande ganho em fotos tortas.
2. **Detecção de região de interesse.** Localizar o bloco de texto com contornos antes do OCR reduz ruído e acelera.
3. **Aprendizado da correção de valor.** Guardar leituras "corrigidas" (antes/depois) para descobrir padrões de erro e automatizar mais correções com segurança.
4. **Modelo de OCR especializado.** Treinar/ajustar um modelo com tickets reais (fine-tune de Tesseract com tesstrain ou um CRNN leve) tende a ser o maior salto de acurácia.
5. **Dashboard de qualidade.** Registrar quantos tickets vieram de cada nível de confiança ao longo do tempo, para acompanhar a melhoria do processo e do lote de fotos.

Como medir melhorias

Antes de mudar qualquer coisa, monte um pequeno conjunto dourado (20–30 tickets com número e valor conferidos à mão) e um script que roda o núcleo sobre ele, comparando resultado x esperado. Sem essa métrica, não há como saber se um ajuste melhorou ou apenas mudou o erro de lugar.

11 GLOSSÁRIO E CONSULTA RÁPIDA

Termos técnicos

Termo	Definição prática
OCR	Optical Character Recognition — conversão de imagem de texto em texto editável.
Binarização	Transformar a imagem em preto e branco; separa texto do fundo.
Otsu	Método que escolhe automaticamente o melhor limiar de binarização.
Abertura morfológica	Erosão seguida de dilatação; remove pontos isolados e suaviza traços.
CLAHE	Equalização de contraste adaptativa com limitação — realça texto apagado regionalmente.
Componente conexo	Bloco de pixels vizinhos com a mesma cor; usado para estimar altura de texto.
IoU	Intersection over Union — medida de sobreposição entre duas caixas (0 a 1).
PSM	Page Segmentation Mode — como o Tesseract interpreta a página; usamos 6 (bloco uniforme).
tessdata	Pasta com os modelos de idioma do Tesseract (ex.: por.traineddata).
GIL	Global Interpreter Lock — limita threads em Python; por isso usamos processos.
onefile	Modo do PyInstaller que gera um único .exe, extraído em pasta temporária ao rodar.

Comandos úteis

Terminal (PowerShell) — do projeto

```
# rodar o script sobre a pasta padrão (./img_ticket)
python extrair_tickets.py

# pasta específica + gerar JSON além do terminal
python extrair_tickets.py "C:\caminho\das\imagens" --json

# abrir com debug mostrando as linhas reconhecidas
python extrair_tickets.py --debug

# abrir a interface gráfica já com uma pasta
python app_tickets.py "C:\caminho\das\imagens"

# gerar o executável (Windows)
pyinstaller --noconfirm TicketsTaxis.spec   # -> dist\TicketsTaxis.exe
```

Mapa do código — onde está cada coisa

Função / símbolo	Arquivo	O que faz
configurar_tesseract()	extrair_tickets.py	localiza tesseract.exe e define TESSDATA_PREFIX
_escala_para_texto()	extrair_tickets.py	estima ampliação para texto de ~60 px
binarizacoes()	extrair_tickets.py	gera as 4 variantes (otsu, open, soft, clahe_open)
ocr_palavras()	extrair_tickets.py	roda OCR nas 4 e deduplica por IoU
agrupar_linhas()	extrair_tickets.py	agrupa palavras em linhas por centro vertical
extrair_numero()	extrair_tickets.py	lê e corrige o nº do ticket

Função / símbolo	Arquivo	O que faz
extrair_valores()	extrair_tickets.py	lê a coluna de valores de cada linha isolada
reler_valor()	extrair_tickets.py	relê o recorte com vários limiares e vota
interpretar()	extrair_tickets.py	regras de negócio → objeto Ticket
processar_lote()	extrair_tickets.py	processa em paralelo com progresso
salvar_excel()	extrair_tickets.py	gera a planilha com fórmula SUM
JanelaTickets._trabalho()	app_tickets.py	thread que chama o núcleo e enfileira eventos
JanelaTickets._drenar_fila()	app_tickets.py	atualiza a interface a partir da fila

Resumo em uma frase

Reaproveitamos um motor de OCR consagrado (Tesseract) e resolvemos a fragilidade dele com **redundância inteligente**: quatro formas de olhar a imagem, consenso por confiança, regras de domínio e a aritmética do próprio recibo como juiz — tudo local, rápido e distribuído como um único .exe.

Documento gerado automaticamente a partir do código-fonte e das imagens processadas pelo próprio sistema. Versões: Python 3.13.2 · OpenCV 5.0.0 · Tesseract 5.5.2 · numpy 2.1.3 · pytesseract 0.3.13 · openpyxl 3.1.5 · Pillow 12.2.0.